贪心

A. Card Game for Two

思路

由于卡片数值 a_i 的总和固定，且双方都采用最优策略，为了最大化自己的收益（即尽可能拿到大的数字），最优策略必然是**每轮取走当前剩余卡片中最大的一张**。

做法：

1. 将所有卡片按数值从大到小排序。
2. Alice 取第 $1, 3, 5, \ldots$ 大的数，Bob 取第 $2, 4, 6, \ldots$ 大的数。
3. 计算两人的得分差值即可。

代码

```cpp
#include <bits/stdc++.h>
#define int long long
#define endl '\n'
#define N 200005
using namespace std;

int n;
int a[N];

signed main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    sort(a + 1, a + 1 + n);
    int res1 = 0, res2 = 0;
    for (int i = n; i >= 1; i--) {
        if ((n - i) & 1)
            res2 += a[i];
        else
            res1 += a[i];
    }
    cout << abs(res1 - res2) << endl;
    return 0;
}
```

B. 凌乱的yyy / 线段覆盖

思路

由于要尽可能选最多的比赛，显然选择右端点越小的，越能为后续留出空间。

因此将线段按照右端点排序后，记当前被覆盖的最大时间为 tim，从左到右按顺序选择 $a_i \geq tim$ 的线段并更新 $tim = b_i$ 即可。

代码

```
#include <bits/stdc++.h>
#define N 1000005
using namespace std;
struct seg {
    int l, r;
}a[N];
signed main() {
    int n; cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i].l >> a[i].r;
    sort(a + 1, a + n + 1, [](auto &x, auto &y) -> bool {
        return x.r < y.r;
    });
    int tim = 0, ans = 0;
    for (int i = 1; i <= n; i++) {
        if (a[i].l >= tim) {
            tim = a[i].r; ans++;
        }
    }
    cout << ans;
    return 0;
}
```

C. Kana and Dragon Quest game

思路

显然，数值大的时候先选择操作一更优。再仔细观察，可以发现先操作一再操作二都是更优的，因为操作二可以降低的值是不变的，而操作一可以降低的值是逐渐下降的。那就贪心的考虑，只要操作一可以减低，就先操作一，否则操作二。

代码

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
ll T,x,n,m,a,b;
int main()
{
        cin>>T;
        while (T--) {
                cin>>x>>n>>m;
                while (n&&x) {
                        n--;
                        x=min(x,x/2+10);
                }
                x=max(0ll,x-10*m);
                if (x) puts("NO");
                else puts("YES");
        }
        return 0;
}
```

D. Cow Cars S

思路

由于每一层会增加一次 D 的速度惩罚，为了让更多奶牛上路，我们应该将速度较慢的奶牛放在惩罚较小的位置，把高速奶牛留给后面惩罚大的位置。

做法：

1. 将所有速度 S_i 从小到大排序。
2. 维护当前已上路奶牛数 `ans` 。对于第 `ans` 个上路的奶牛，它处于第 `ans / M` 层，受到的惩罚是 `(ans / M) * D` 。
3. 遍历排序后的奶牛，如果当前奶牛速度 S_i 减去当前惩罚仍不小于 L，则该奶牛可以上路， `ans++` ；否则该奶牛太慢，直接舍弃。

代码

```cpp
#include <bits/stdc++.h>
#define int long long
#define endl '\n'
#define N 50005
using namespace std;

int n, m, d, l;
int a[N];

signed main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    cin >> n >> m >> d >> l;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    sort(a + 1, a + 1 + n);
    int ans = 0;
    for (int i = 1; i <= n; i++)
        if (a[i] - (ans / m) * d >= l)
            ans++;
    cout << ans << endl;
    return 0;
}
```

E. Smallest Subsequence（★5）

思路

由于要使字典序最小，在前 $i-1$ 位不变的情况下，如果能将第 i 位替换为更小的字符一定更优。

因此可用单调栈来实现选择字符的过程：若栈顶元素大于当前字符，则出栈，直到栈为空或栈顶元素小于等于当前字符，这样每次操作过后得到的字典序一定在当前扫描位置下最优。

需要注意的是因为需要得到长度为 k 的子序列，如果继续将栈内元素出栈会导致后续字符数不够，则不能出栈。具体体现为判定 $siz+n-i+1>k$ 时才可出栈。

代码

```
#include <bits/stdc++.h>
#define N 100005
using namespace std;
char st[N];
signed main() {
    cin.tie(nullptr) -> sync_with_stdio(false);
    int n, k, siz = 0; cin >> n >> k;
    string s; cin >> s; s = ' ' + s;
    for (int i = 1; i <= n; i++) {
        while (siz && siz + n - i + 1 > k && st[siz] > s[i]) siz--;
        if (siz < k) st[++siz] = s[i];
    }
    cout << st + 1;
    return 0;
}
```

F. Little Girl and Maximum XOR

思路

从高位往低位考虑，当该位上两数相等，该位异或值一定为0；到该位上两数不等时，可以取该位为1后面为0和该位为0后面为1的两个数，使得异或值为从该位置开始全1，所以答案为$2^{\text{从高到低第一个不同的位置}+1}-1$。

代码

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
ll l,r,ans;
int main()
{
        scanf("%lld%lld",&l,&r);
        for (int i=62;i>=0;i--)
                if (((l>>i)&1)!=((r>>i)&1)) {
                        ans=(1ll<<(i+1))-1;
                        break;
                }
        printf("%lld",ans);
        return 0;
}
```

G. Changing the String

思路

为了让字符串字典序最小，我们从左到右遍历，对每个字符贪心处理：

1. **优先变 a**：直接变换总是优于间接变换，因为它省下了一次“中转”操作供后续使用。
   - 首选直接变换 ($x \to a$)，只消耗 1 个操作。
   - 次选间接变换 ($x \to y \to a$)，消耗 2 个操作，且要求操作下标 $i < j$。
2. **其次变 b**：如果当前是 c 则尝试变成 b。

使用 `std::set` 维护各类操作的下标。

代码

```cpp
#include <bits/stdc++.h>
#define int long long
#define endl '\n'
using namespace std;

int n, q;
string s;
set<int> S[3][3];  // S[u][v] 存储 u->v 的所有操作下标

signed main() {
    cin.tie(0)->sync_with_stdio(0);
    int T;
    cin >> T;
    while (T--) {
        cin >> n >> q >> s;
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                S[i][j].clear();
        for (int i = 1; i <= q; i++) {
            char u, v;
            cin >> u >> v;
            S[u - 'a'][v - 'a'].insert(i);
        }
        for (auto& c : s) {
            if (c == 'a')
                continue;
            int u = c - 'a';
            // 直接 u -> a
            if (S[u][0].size()) {
                S[u][0].erase(S[u][0].begin());
                c = 'a';
            }
            // 间接 u -> mid -> a
            else {
                int mid = 3 - u;
                if (S[u][mid].size() && S[mid][0].size()) {
                    int id1 = *S[u][mid].begin();
                    auto it = S[mid][0].upper_bound(id1);  // 找 id2 > id1
                    if (it != S[mid][0].end()) {
                        S[u][mid].erase(id1);
                        S[mid][0].erase(it);
```

```
                    c = 'a';
                }
            }
        }
        // c -> b
        if (c == 'c' && !S[2][1].empty()) {
            S[2][1].erase(S[2][1].begin());
            c = 'b';
        }
    }
    cout << s << endl;
}
return 0;
}
```

H. 合并果子

思路

我们将 a_1 与 a_2 合并，产生的代价为 $a_1 + a_2$，将合并后的堆再与 a_3 合并时，产生的代价为 $a_1 + a_2 + a_3$，将合并后的堆再与 a_4 合并，产生的代价为 $a_1 + a_2 + a_3 + a_4$，以此类推。

由此可以看出，一堆石子在整个过程中产生的总代价为 $a_i \times N$ 其中 N 为其所在的堆参与合并的总次数。因此尽量让数量少的堆合并次数更多，即每次优先选取最小的两个堆进行合并。

这一过程可用优先队列进行维护。

代码

```
#include <bits/stdc++.h>
using namespace std;
signed main() {
    int n; cin >> n;
    priority_queue<int, vector<int>, greater<int>> q;
    for (int i = 1; i <= n; i++) {
        int x; cin >> x;
        q.push(x);
    }
    int ans = 0;
    for (int i = 1; i < n; i++) {
        int x = q.top(); q.pop();
        int y = q.top(); q.pop();
        ans += x + y;
        q.push(x + y);
    }
    cout << ans;
    return 0;
}
```

I. 加工生产调度

思路

求一个加工顺序使得加工总用时最短，就是让机器的空闲时间最短。A车间从开始到做完前是不会停止的，所以我们的目标就是让B车间空闲时间最短。
贪心的考虑，让A车间上加工时间短的产品先加工，这样使得B车间能在尽早开始加工；把在B车间上加工时间短的产品放在后面加工，这样使得B车间用尽可能长的时间等待A车间完工，从而避免无工可作。

代码

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
ll n,l,r,suma,sumb,a[10000],b[10000],q[10000];
struct node
{
	ll v,id;
	inline bool operator < (const node& x) const {
		return v<x.v;
	}
}c[10000];
int main()
{
	scanf("%lld",&n);
	for (int i=1;i<=n;i++) {
		scanf("%lld",&a[i]);
		c[i].v=a[i];
		c[i].id=i;
	}
	for (int i=1;i<=n;i++) {
		scanf("%lld",&b[i]);
		c[i].v=min(c[i].v,b[i]);
	}
	sort(c+1,c+n+1);
	l=1;
	r=n;
	for (int i=1;i<=n;i++) {
		if (c[i].v==a[c[i].id]) q[l++]=c[i].id;
		else q[r--]=c[i].id;
	}
	for (int i=1;i<=n;i++) {
		suma+=a[q[i]];
		sumb=max(suma,sumb)+b[q[i]];
	}
	printf("%lld\n",sumb);
	for (int i=1;i<=n;i++)
		printf("%lld ",q[i]);
	return 0;
}
```

J. 国王游戏

思路

1. 假设相邻两人为 (L_1, R_1) 和 (L_2, R_2)，排在他们前面所有人的左手乘积为 P。
   - 若不交换，两人获得的最大奖赏为 $\max(\frac{P}{R_1}, \frac{P \times L_1}{R_2})$。
   - 若交换，两人获得的最大奖赏为 $\max(\frac{P}{R_2}, \frac{P \times L_2}{R_1})$。
2. 现在需要比较 $\max(\frac{P}{R_1}, \frac{P \times L_1}{R_2})$ 和 $\max(\frac{P}{R_2}, \frac{P \times L_2}{R_1})$。
   - 首先约去 P。
   - 注意到，$\frac{L_1}{R_2}$ 肯定大于 $\frac{1}{R_2}$，$\frac{L_2}{R_1}$ 肯定大于 $\frac{1}{R_1}$。所以式子中的最大值主要取决于 $\frac{L_1}{R_2}$ 和 $\frac{L_2}{R_1}$。
   - 若要不交换 < 交换，则需要满足：$\frac{L_1}{R_2} < \frac{L_2}{R_1}$，即：$L_1 \times R_1 < L_2 \times R_2$。
   - 所以为了让最大奖赏最小，我们应该按照 $L \times R$ **从小到大** 的顺序排列大臣。

> 由于乘积 P 会变得非常大（最坏情况 10000^{1000}），`long long` 存不下，必须使用**高精度算法**（见如下代码）。

代码

```
#include <bits/stdc++.h>
#define int long long
#define endl '\n'
#define N 1005
using namespace std;

// 高精度模板
struct BigInt {
    vector<int> A;

    /// 构造函数
    BigInt(int n = 0) {
        if (n == 0)
            A.push_back(0);
        while (n) {
            A.push_back(n % 10);
            n /= 10;
        }
    }

    // 去除前导零
    void trim() {
        while (A.size() > 1 && A.back() == 0)
            A.pop_back();
    }

    // 高精小于号
    bool operator<(const BigInt& b) const {
        if (A.size() != b.A.size())
            return A.size() < b.A.size();
        for (int i = A.size() - 1; i >= 0; i--)
            if (A[i] != b.A[i])
                return A[i] < b.A[i];
        return 0;
    }

    // 高精 + 高精
    BigInt operator+(const BigInt& b) {
        BigInt C;
        C.A.clear();
        int t = 0;
```

```
    for (int i = 0; i < A.size() || i < b.A.size() || t; i++) {
        if (i < A.size())
            t += A[i];
        if (i < b.A.size())
            t += b.A[i];
        C.A.push_back(t % 10);
        t /= 10;
    }
    return C;
}

// 高精 - 高精
BigInt operator-(const BigInt& b) {
    BigInt C = *this;
    int t = 0;
    for (int i = 0; i < b.A.size() || t; i++) {
        t = C.A[i] - t - (i < b.A.size() ? b.A[i] : 0);
        C.A[i] = (t + 10) % 10;
        if (t < 0)
            t = 1;
        else
            t = 0;
    }
    C.trim();
    return C;
}

// 高精 * 高精
BigInt operator*(const BigInt& b) {
    BigInt C;
    C.A.assign(A.size() + b.A.size(), 0);
    for (int i = 0; i < A.size(); i++) {
        for (int j = 0; j < b.A.size(); j++) {
            C.A[i + j] += A[i] * b.A[j];
            C.A[i + j + 1] += C.A[i + j] / 10;
            C.A[i + j] %= 10;
        }
    }
    C.trim();
    return C;
}

// 高精 / 低精
```

```
    BigInt operator/(const int& b) {
        BigInt C;
        C.A.resize(A.size());
        int r = 0;
        for (int i = A.size() - 1; i >= 0; i--) {
            r = r * 10 + A[i];
            C.A[i] = r / b;
            r %= b;
        }
        C.trim();
        return C;
    }

    void print() {
        for (int i = A.size() - 1; i >= 0; i--)
            cout << A[i];
        cout << endl;
    }
};

struct Person {
    int l, r;
    bool operator<(const Person& b) const { return l * r < b.l * b.r; }
} a[N];
int n, L, R;

signed main() {
    cin.tie(0)->sync_with_stdio(0);
    cin >> n >> L >> R;
    for (int i = 1; i <= n; i++)
        cin >> a[i].l >> a[i].r;
    sort(a + 1, a + 1 + n);
    BigInt P(L);  // 当前前面所有人的左手乘积
    BigInt ans(0);
    for (int i = 1; i <= n; i++) {
        BigInt cur = P / a[i].r;
        if (ans < cur)
            ans = cur;
        P = P * BigInt(a[i].l);
    }
    ans.print();
    return 0;
}
```

K. Serval and Parenthesis Sequence

思路

一般的括号匹配思路：首先判断左右括号数量是否相等，不等则一定不合法。然后记 cnt 为待匹配的左括号数量，从左往右遍历字符串，遇到左括号则令$cnt+1$，遇到右括号则令$cnt-1$。若中途出现 $cnt<0$ 的情况则不合法。

本题中为了使得前缀中不出现合法括号序列，则不能在中途出现 $cnt=0$ 的情况（左括号全部和右括号匹配），因此尽量优先将?变为左括号以增加 cnt，但由于要求整个字符串为合法括号序列，因此当左括号数量已经达到 $n/2$ 时，只能增加右括号了。

代码

```
#include <bits/stdc++.h>
using namespace std;
signed main() {
    int n; cin >> n;
    if (n % 2) {cout << ":("; return 0;}
    string s; cin >> s;
    int cnt = 0, siz = 0;
    for (int i = 0; i < n; i++)
        if (s[i] == '(') cnt++;
    if (cnt > n / 2) {cout << ":("; return 0;}
    for (int i = 0; i < n; i++) {
        if (s[i] == '(') siz++;
        else if (s[i] == '?' && cnt < n / 2) {
            s[i] = '('; siz++; cnt++;
        } else {
            if (!siz) {cout << ":("; return 0;}
            s[i] = ')'; siz--;
        }
        if (!siz && i < n - 1) {cout << ":("; return 0;}
    }
    cout << s;
    return 0;
}
```

L. Minimal string

思路

1. 将字符串每个元素依次入栈；
2. 在当前栈顶元素小于等于任何未入栈元素时弹栈（因为该答案位置不存在更优秀的字符，字符串元素小于等于该字符，栈里的元素在底下也不可能弹出）；
3. 在所有元素都入栈后，弹空栈。

> 注意需要O(n)预处理第i个字符串位置后面的最大的元素。

代码

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
ll n;
char ch,s[100010],mn[100010];
stack<char>stk;
int main()
{
        cin>>s;
        n=strlen(s);
        mn[n]='z'+1;
        for (int i=n-1;i>=0;i--)
                mn[i]=min(mn[i+1],s[i]);
        for (int i=0;i<n;i++) {
                stk.push(s[i]);
                while (!stk.empty()) {
                        ch=stk.top();
                        if (ch>mn[i+1]) break;
                        putchar(ch);
                        stk.pop();
                }
        }
        while (!stk.empty()) {
                ch=stk.top();
                putchar(ch);
                stk.pop();
        }
        return 0;
}
```

M. Pavel and Triangles

思路

1. **三角形性质**:
   - 题目中给定的木棍长度都是 2 的幂（$1, 2, 4, 8 \ldots$）。
   - 根据三角形不等式 $a + b > c$，如果最大边是 2^k，那么另外两边之和必须大于 2^k。

- 由于 $\sum_{i=0}^{k-1} 2^i = 2^k - 1 < 2^k$，这意味着如果只用小于 2^k 的边，那么永远无法和一条 2^k 组成三角形。
- 所以得到结论：合法的三角形必须包含**至少两条**长度为 2^k 的最长边。即形式必须为 $(2^k, 2^k, 2^j)$，其中 $j \le k$。

2. **贪心**:
   我们从小到大遍历每种长度。为了最大化数量，我们应该“用最小的代价填坑”。维护一个变量 `rem` ，表示从之前的层级遗留下来、可以作为第三条边的废弃木棍数量。对于当前长度 2^i 的木棍：
   i. 优先组装混合三角形，用 `rem` 中的一根小木棍，搭配两根当前的 2^i，组成 $(2^i, 2^i, \text{small})$。这只消耗 2 根当前木棍。
   ii. 其次组装等边三角形，如果 `rem` 用完了或当前木棍还有剩，就用三根 2^i 组成 $(2^i, 2^i, 2^i)$。
   iii. 剩下的不能成组的 2^i 木棍加入 `rem` ，供以后的更长边使用。

代码

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, x;

signed main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    cin >> n;
    int ans = 0, rem = 0;
    for (int i = 1; i <= n; i++) {
        cin >> x;
        int use = min(rem, x >> 1);  // 优先用以前剩下的废棍做第三边
        // x >> 1 等价于 x / 2
        ans += use;     // 组成 (2^i, 2^i, small)
        rem -= use;     // 消耗废棍
        x -= use << 1;  // 消耗当前木棍
        // x << 1 等价于 x * 2
        ans += x / 3;  // 剩下的自己组成 (2^i, 2^i, 2^i)
        rem += x % 3;  // 组不完的变成新的废棍
    }
    cout << ans << endl;
    return 0;
}
```

N. Maximum XOR Sum

思路

朴素的从高位到低位按收益贪心地选取0或1，可能因高位贴合 K 而导致低位无法选取高收益值。本题采用**带反悔的贪心**：维护一个变量 dif，记录“如果在之前某位脱离 K 限制所需的最小代价”。当遇到因为高位贴合 K 而被迫填 `0` 且亏损巨大的情况时，通过比较决定是否反悔。

从高位向低位遍历，维护标记 f (是否受 K 限制) 和 dif (脱离限制的最小代价)。

1. **已自由 (`!f`)**：
   不受 K 限制，直接取 `0` 或 `1` 中收益较大的那个累加。
2. **仍受限 (`f`)**：
   若 K 当前位为 `1`：
   - **选 `0` 更优**：直接选 `0`，状态转变为自由 (`f=0`)。
   - **选 `1` 更优**：暂选 `1` (保持受限)。更新 $dif = \min(dif, \text{本位选0造成的损失})$。
   若 K 当前位为 `0`：
   - **选 `0` 更优**：直接选 `0`。
   - **选 `1` 更优**：比较当前位亏损 d 与反悔代价 dif。若 $dif \le d$，执行反悔：获得 $d - dif$ 的收益，状态变为自由。否则令 $dif = dif - d$（当前位置如果能够选 `1` 将弥补一部分亏损）

代码

```
#include<bits/stdc++.h>
#define N 100005
#define int long long
using namespace std;
int a[N], cnt[N];
signed main() {
    cin.tie(nullptr) -> sync_with_stdio(false);
    int n, k, ck = 0; cin >> n >> k;
    while(k) {
        a[++ck] = k & 1;
        k >>= 1;
    }
    int mx = 0;
    for(int i = 1; i <= n; i++) {
        int x, now = 0; cin >> x;
        while(x) {
            cnt[++now] += x & 1;
            x >>= 1;
        }
        mx = max(mx, now);
    }
    int ans = 0, dif = 2e18;
    for(int i = mx; i > ck; i--)
        ans += cnt[i] * (1ll << (i - 1));
    bool f = 1;
    for(int i = ck; i; i--) {
        if(!f) ans += max(cnt[i], n - cnt[i]) * (1ll << (i - 1));
        else if(a[i] == 1) {
            if(cnt[i] >= n - cnt[i]) {
                f = 0;
                ans += cnt[i] * (1ll << (i - 1));
            } else {
                ans += (n - cnt[i]) * (1ll << (i - 1));
                dif = min(dif, (n - cnt[i] * 2) * (1ll << (i - 1)));
            }
        } else {
            ans += cnt[i] * (1ll << (i - 1));
            if(cnt[i] < n - cnt[i]) {
                int d = (n - cnt[i] * 2) * (1ll << (i - 1));
                if(dif <= d) {
                    ans += d - dif;
```

```
                    f = 0;
                } else {
                    dif -= d;
                }
            }
        }
    }
    cout << ans << endl;
    return 0;
}
```

O. 种树

思路

将序列视为“峰-谷-峰-谷…”的交替结构，每次选当前最大的“峰”最优。选中一个峰后，它两侧的“谷”就再也不可能被选（与之相邻），把这两个谷“合并”后，左右两侧原来的相邻“峰”会连成一个更大的候选峰。
用一个大根堆维护当前所有元素的“贡献值”，用双向链表支持把被“吃掉”的两侧元素从链表中删除；每次取出堆顶的最大值，如果是负的就可以停止（继续选只会让答案更小）。
反悔贪心，当我们选中位置 x（贡献$a[x]$）后，要把它两侧的元素$l[x]$和$r[x]$删除，并把 x 作为新的“合并点”留在链表里，更新它的贡献为$a[x] \leftarrow a[l[x]] + a[r[x]] - a[x]$,下次如果再选中$x$,相当于抵消了这次的选择，改为选择$l[x]$和$r[x]$。

代码

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
ll n,k,x,ans,a[300010],v[300010],l[300010],r[300010];
struct node
{
	ll v,id;
	inline bool operator < (const node& x) const {
		return v<x.v;
	}
}now;
priority_queue<node>q;
inline void del(ll x)
{
	l[r[x]]=l[x];
	r[l[x]]=r[x];
}
int main()
{
	scanf("%lld%lld",&n,&k);
	for (int i=1;i<=n;i++) {
		scanf("%lld",&a[i]);
		l[i]=i-1;
		r[i]=i+1;
		q.push(node{a[i],i});
	}
	while (k--) {
		now=q.top();
		q.pop();
		while (v[now.id]) {
			now=q.top();
			q.pop();
		}
		x=now.id;
		if (now.v<0) break;
		ans+=now.v;
		a[x]=a[l[x]]+a[r[x]]-a[x];
		q.push(node{a[x],x});
		v[l[x]]=v[r[x]]=1;
		del(l[x]);
		del(r[x]);
```

```
        }
        printf("%lld",ans);
        return 0;
}
```

P. Warehouse Store

思路

为了让接受的订单数量最大，我们需要让消耗的库存尽可能少。也就是**优先满足需求量 b_i 小的订单**。直接贪心是不行的，因为当前一个大订单可能会耗尽库存，导致后面多个小订单无法完成。所以使用反悔贪心。

1. 每天先累加进货量 a_i 到库存。
2. 假设接受当前的订单 b_i：扣除库存，将该订单放入大根堆。
3. 如果此时库存为负，说明资源不够了。为了保证“接受数量最大化”且“库存剩余最大化”，我们在已接受的订单中（包括刚放进去的），**反悔那个需求量最大**的订单。这样做相当于：要么不接受当前这个大订单；要么撤销以前一个更大的订单来换取当前这个较小的订单，腾出更多库存。
4. 最后堆中保留的订单即为最优解。

代码

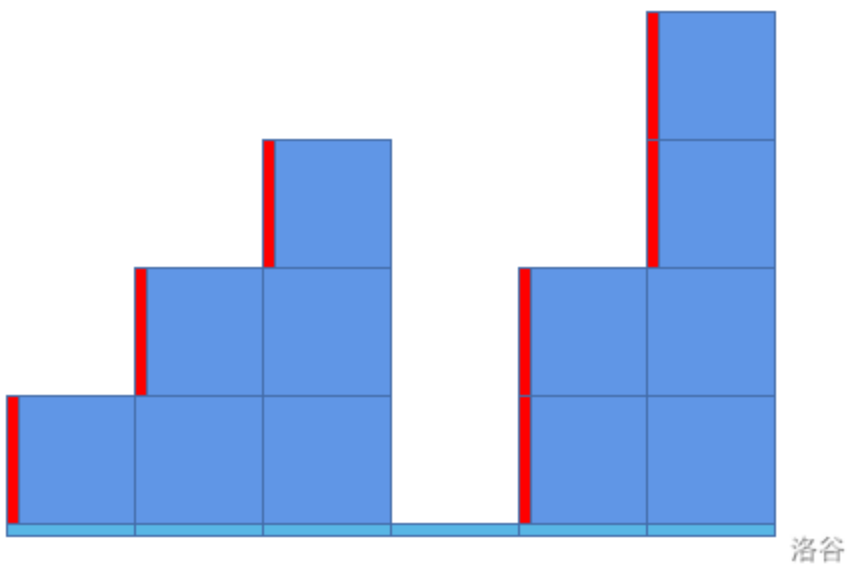
```cpp
#include <bits/stdc++.h>
#define int long long
#define endl '\n'
#define N 250005
#define pii pair<int, int>
using namespace std;

int n;
int a[N], b[N];

signed main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    for (int i = 1; i <= n; i++)
        cin >> b[i];
    priority_queue<pii> q;  // 大根堆存储 {需求量, 订单编号}
    int t = 0;
    for (int i = 0; i < n; i++) {
        t += a[i];  // 进货
        t -= b[i];  // 尝试接单
        q.push({b[i], i + 1});
        // 如果爆仓，踢掉需求最大的那个单子
        if (t < 0) {
            t += q.top().first;
            q.pop();
        }
    }
    cout << q.size() << endl;
    vector<int> ans;
    while (q.size()) {
        ans.push_back(q.top().second);
        q.pop();
    }
    sort(ans.begin(), ans.end());
    for (int i = 0; i < ans.size(); i++)
        cout << ans[i] << ' ';
    return 0;
}
```

Q. Ina of the Mountain

思路

首先将题目进行一定的转化，注意到对于一个章鱼（数字）a_i，我们可以执行 $(a_i \bmod k) + mk$ 次操作来使其符合条件（n 为非负整数）。当确定了每个章鱼需要被执行的操作次数后，我们便可以容易地得出答案。

假设确定了每个章鱼需要被执行的操作次数为 $[1, 2, 3, 0, 2, 4]$，如下图所示，总共需要执行的操作次数即为红线标出的总长度，即 $1 + 1 + 1 + 2 + 2 = 7$。

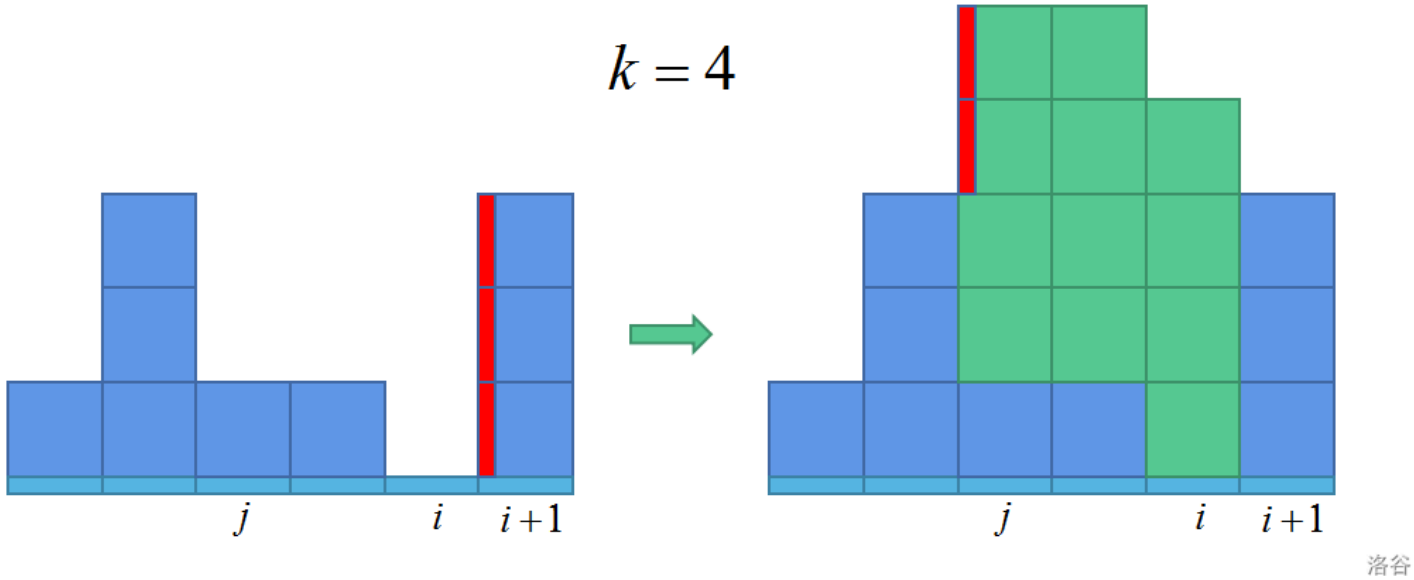


不妨将每个章鱼的取值看作**高度**，将一个章鱼需要被操作的次数加上 k 称作**提升**。同时，由题意，高度恰好等于 k 的章鱼可以视为高度为 0，在下文中默认将所有高度恰好为 k 的章鱼的高度看作 0。

假设已经确定了 i 个章鱼需要被操作的次数，考虑第 $i + 1$ 个章鱼，将第 i 个章鱼的高度记作 h_i。

1. 若 $h_{i+1} \le h_i$，那么可以直接不做任何提升操作。
2. 若 $h_{i+1} > h_i$，有以下两种选择：
   - 不做任何提升操作，将答案增加 $h_{i+1} - h_i$；
   - 选择一个 $j \le i$，使得 $h_j + k - h_{j-1} - \max(0, h_j - h_{j-1})$ 最小（这个最小值后文记作 Δ_i），并提升 $[j, i]$ 范围中的每个章鱼，将答案增加 Δ_i。

第一种操作很好理解，第二种操作的思想是将新增的章鱼前的章鱼提升，使得新增的章鱼不对答案产生贡献。同时，容易得知，如果选定一个章鱼，并将其及其后所有章鱼提升，其后章鱼产生的贡献不变，只有被选定的章鱼贡献产生变化。可参考下图。



在两种操作中选择一个贡献较小的，就完成了第 $i+1$ 个章鱼的计算。

然而，每新增一个章鱼，这种方法就要向前遍历每一个章鱼，时间复杂度为 $O(n^2)$，无法通过此题。

优先队列优化：

注意到，在上面的方法中，每个 j 最多被选中一次，且选择任意一个 j 进行操作后都不会影响其他任意一个 Δ_i。那么，在每次新增一个章鱼后，我们可以预先计算好这个章鱼的 Δ_i，并将其压入一个优先队列，这样之后的查询可以以 $O(1)$ 的复杂度完成。同时，由于每个 j 最多被选中一次，当我们通过优先队列获取到对应的 j 后，只需要将其出队而无需重新计算并入队。

代码

```
#include<bits/stdc++.h>
#define N 200005
#define int long long
using namespace std;
int a[N];
signed main() {
    cin.tie(nullptr) -> sync_with_stdio(false);
    int t; cin >> t;
    while(t--) {
        priority_queue<int, vector<int>, greater<int>> q;
        int n, k, ans = 0; cin >> n >> k;
        for(int i = 1; i <= n; i++) {
           cin >> a[i]; a[i] %= k;
        }
        ans += a[1];
        for(int i = 2; i <= n; i++) {
            if(a[i] > a[i - 1]) {
                int x = q.empty() ? k : q.top();
                if(a[i] - a[i - 1] > x) {
                    q.pop(); a[i - 1] += k;
                    ans += x;
                } else {
                    ans += a[i] - a[i - 1];
                }
            }
            if(a[i] < a[i - 1]) q.push(a[i] + k - a[i - 1]);
        }
        cout << ans << '\n';
    }
    return 0;
}
```

R. Teleporter

思路

原图是一个函数图（每点出度为 1）。由于“任意点能到达 1”，整个图中唯一的环必然包含点 1。
若$a[1] \neq 1$，则 1 所在的环长度 > 1。为了让“第 K 次传送在 1”，把 $a[1]$ 改成 1 是最佳且必要的一步（使 1 成为自环，之后在 1 停留即可）。

接下来忽略点 1 的出边，把边视为无向边连 i 与 $a[i]$（$i \geq 2$）。这样就得到以 1 为根的一棵树（因为删去 1 的那条出边后，原来的唯一环被打断）。
在这棵以 1 为根的树上，希望对某些点 x 把 $a[x]$ 改成 1，使得对每个结点 y：

1. 若 y 到 1 的原始距离 $dep(y) \geq K$，则 y 向上走 $K-1$ 步到达某个祖先 x，且“第 K 步”从 x 必须直接到 1（即 $a[x]=1$）；否则第 K 步不会落在 1。
2. 若 $dep(y) < K$，不需要改（y 会提前到 1，之后在 1 自环上停留，到第 K 步仍在 1）。

代码

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
ll n,k,cnt,ans,a[1000010],dep[1000010],head[1000010];
struct node
{
	ll to,next;
}e[1000010];
inline void add(ll x,ll y)
{
	e[++cnt].next=head[x];
	e[cnt].to=y;
	head[x]=cnt;
}
inline void dfs(ll x,ll fa)
{
	dep[x]=1;
	for (int i=head[x];i;i=e[i].next) {
		ll y=e[i].to;
		if (y==fa) continue;
		dfs(y,x);
		if (x!=1&&dep[y]==k) {
			ans++;
			dep[y]=0;
		}
		dep[x]=max(dep[x],dep[y]+1);
	}
}
int main()
{
	scanf("%lld%lld",&n,&k);
	for (int i=1;i<=n;i++)
		scanf("%lld",&a[i]);
	if (a[1]!=1) {
		ans++;
		a[1]=1;
	}
	for (int i=2;i<=n;i++) {
		add(a[i],i);
		add(i,a[i]);
	}
```

```
        dfs(1,0);
        printf("%lld",ans);
        return 0;
}
```

S. Reverse Maximisation

思路

我们需要找到一个 $X \leq N$，使得 X 翻转后的值最大。
为了让翻转后的值最大，我们需要 X 的末尾（低位）尽可能大（即全是 9）。

对于输入的数字串 N，我们只需要比较 3 种候选情况：

1. N 本身
2. 将第 1 位数字减 1，后面全部变为 9。（例如 $460 \to 399$）。
3. 如果 N 的第 1 位是 1，我们还可以尝试保持第 1 位不变，找到第一个非 0 的后续位置 k，将其减 1，后面全变 9。（例如 $183 \to 179$）。

只需比较这 3 个候选数对应的 $\mathrm{Reverse}(X)$ 值即可。

代码

```
#include <bits/stdc++.h>
#define int long long
#define endl '\n'
using namespace std;

// 去除前导零
string trim(string s) {
    s.erase(0, min(s.find_first_not_of('0'), s.size() - 1));
    return s;
}

// 翻转
string rev(string s) {
    s = trim(s);
    reverse(s.begin(), s.end());
    return trim(s);
}

// 比较
bool larger(const string& a, const string& b) {
    if (a.size() != b.size())
        return a.size() > b.size();
    return a > b;
}

signed main() {
    cin.tie(0)->sync_with_stdio(0);
    int T;
    cin >> T;
    while (T--) {
        string s;
        cin >> s;
        vector<string> v = {s};  // 候选
        // 修改第0位
        string s2 = s;
        if (s2[0] > '0') {
            s2[0]--;
            for (int i = 1; i < s2.size(); i++)
                s2[i] = '9';
            v.push_back(s2);
        }
```

```
        // 如果首位是1，修改第一个非0的后缀位
        if (s.size() > 1 && s[0] == '1') {
            string s3 = s;
            int k = 1;
            while (k < s3.size() && s3[k] == '0')
                k++;
            if (k < s3.size()) {
                s3[k]--;
                for (int i = k + 1; i < s3.size(); i++)
                    s3[i] = '9';
                v.push_back(s3);
            }
        }
        string ans = "", max = "";
        for (auto& x : v) {
            string val = rev(x);
            if (ans == "" || larger(val, max)) {
                max = val;
                ans = trim(x);
            }
        }
        cout << ans << endl;
    }
    return 0;
}
```

T. Mr. Kitayuta vs. Bamboos

思路

由于题目是一个要最小化最大值的问题，不难想到使用二分答案。

考虑对于一个答案 H 如何验证可行性。我们可以将整个过程逆向处理：将每个竹子的最终高度设定为 H，每天降低 a_i，而操作变为将其高度拔高 p，判定能否在没有竹子中途高度小于0的情况下，使每根竹子回溯到的初始高度均大于等于 h_i 即可。

考虑操作的策略：

1. **判定每根竹子是否需要拔高：** 当前高度在回溯到初始状态时是否大于等于初始高度，即 $H + cnt_i * p - a_i * m \geq h_i$ ，其中 cnt_i 表示该竹子已被拔高的次数，不满足条件时才需要再拔高。

2. **规划拔高的次序：** 显然应当优先拔高会在最早的天数内高度小于0的竹子，即 $(H + cnt_i * p)/a_i$ 最小，我们使用优先队列进行维护，如果发现堆顶的竹子所需的最晚日期已经小于了当前日期，说明来不及拔高这根竹子，判定 H 不可行。

代码

```
#include<bits/stdc++.h>
#define N 100005
#define int long long
using namespace std;
int n, m, k, p;
int c[N], a[N], h[N];
struct node {
    int v, id;
    bool operator < (const node &u) const{return v > u.v;}
};
bool judge(int x) {
    priority_queue<node> q;
    for(int i = 1; i <= n; i++) {
        if(x - a[i] * m < h[i])
            q.push({x / a[i], i});
        c[i] = 0;
    }
    for(int i = 1; i <= m; i++)
        for(int j = 1; j <= k; j++) {
            if(q.empty()) return 1;
            node t = q.top(); q.pop();
            if(t.v < i) return 0;
            c[t.id]++;
            if(x + c[t.id] * p - a[t.id] * m < h[t.id]) {
                q.push({(x + c[t.id] * p) / a[t.id], t.id});
            }
        }
    return q.empty();
}
signed main() {
    cin.tie(nullptr) -> sync_with_stdio(false);
    cin >> n >> m >> k >> p;
    int l = 0, r = 0;
    for(int i = 1; i <= n; i++) {
        cin >> h[i] >> a[i];
        r = max(r, h[i] + a[i] * m);
    }
    while(l < r) {
        int mid = l + r >> 1;
        if(judge(mid)) r = mid;
        else l = mid + 1;
```

```
    }
    cout << r << endl;
    return 0;
}
```